Instructor Notes

Programming 101 for FIRST Tech Challenge · Version 2
Companion to **FTC_Programming_101_v2.pdf** (58 slides)

> **How to use these notes.** One entry per slide, in order. Each entry gives the slide title, a suggested time, what to say, a question to ask the class, and anything to watch out for. Times are a guide for a first delivery — total them up and plan your sessions around the natural break points marked in the notes.

> **You do not need to be a programmer to deliver this.** Everything on the slides is sourced and cited, and the notes tell you what the point of each slide is. Where a question has a right answer, the answer is given in brackets. If a student asks something the notes do not cover, “I don't know, let's look it up” is a good answer — and modelling that is part of the lesson.

> **The single best thing you can do** is run the questions. Students who have said an idea out loud remember it; students who have only heard it do not. Where a note says to demonstrate something physically or on a whiteboard, that is worth more than the extra minutes it costs.

Part 0 — The loop

1 Title slide

Opening slide. 1 min.

Say: Set the frame before anything else: nobody here is expected to already know this, and nobody will be asked to memorise it. Say out loud that Pedro Pathing is a separate deck for later, so students do not worry about the drive code today.

Ask the class: Ask: who has written any code before, in any language, even Scratch? Get a show of hands so you know your room.

---

2 Six stops. Then you can read robot code.

Roadmap. 2 min.

Say: Walk the five parts quickly. Emphasise the preview-then-recap pattern: every part shows its vocabulary at the start and again at the end. Tell them the recap slide is the one to photograph.

Ask the class: Ask: which of these five words do you already half-know?

Watch out: Do not linger. This slide is signposting, not teaching.

---

3 Your robot is a goldfish with excellent reflexes.

The single most important slide in the deck. 5 min.

Say: Everything later depends on this. Say it plainly: the robot has no memory of its own plan; it just repeats read-decide-act about fifty times a second. Reference this slide again in Part 3 (state machines) and Part 4 (OpMode). Count to five out loud and tell them the robot just did that 250 times.

Ask the class: Ask: if the loop runs 50 times a second, how many times does it run in a 2 minute 30 second match? (About 7,500.)

Watch out: Students often assume the robot 'runs the program top to bottom once'. Correct that here or it causes confusion for the rest of the session.

---

4 Here is a real program. You cannot read it yet.

The promise. 3 min.

Say: Do not explain any of the code. That is deliberate. Let them look at it and feel slightly lost, then promise the payoff. Point at one line at random and say 'you will be able to explain that one'.

Ask the class: Ask: point at the one line you think you can already guess the meaning of. Usually somebody picks the telemetry line.

Watch out: Resist the urge to start explaining. The whole device fails if you spoil it now.

Part 1 — Java

5 Section opener — Java

Section opener. 1 min.

Say: Read the index aloud. Tell them this is the longest part and the most boring, and that it gets much more interesting in Part 2.

Watch out: Set expectations honestly. Vocabulary sections are a slog; saying so builds trust.

---

6 Sixteen words you are about to meet.

Preview. 2 min.

Say: Read the list. Do not explain anything. The purpose is familiarity, so that when a word appears in ten minutes it is already slightly known rather than brand new.

Ask the class: Ask: which of these have you seen before, anywhere?

Watch out: Two minutes maximum. This is a trailer, not the film.

---

7 One class, one file. Same name, every time.

New in v2. 4 min.

Say: This answers the question students hit in their first ten minutes in Android Studio: 'where do I put this?'. The rule is one public class per file, file named after the class. Mention that packages are just folders, so the package line is not mysterious.

Ask the class: Ask: if I make a class called Drivetrain, what must the file be called? (Drivetrain.java, capital D.)

Watch out: The enum exception matters, because slide 14 and slide 38 both show an enum written inside a class. Flag it now so it does not look like a contradiction later.

---

8 A variable is a box with a name taped on the front.

4 min.

Say: The box analogy carries the whole slide. Emphasise that the type comes first and is not optional, which is the thing that surprises anyone who has used Scratch or Python.

Ask the class: Ask: what type would I use for 'how many game pieces are we holding'? (int.) For 'how fast is the flywheel'? (double.)

Watch out: camelCase is worth enforcing from day one. It is a free habit now and expensive to fix in March.

---

9 Java has eight basic types. You will use three.

3 min.

Say: Do not list all eight types. Three plus String is the whole working set for FTC. The capital S on String is a useful hook forward to Part 2.

Ask the class: Ask: motor power runs from -1.0 to 1.0. Which type? (double. If they say int, ask what int would do to 0.5 — it cannot hold it.)

---

10 Curly braces are walls. Variables cannot climb them.

4 min. Quietly one of the most important slides.

Say: The red box is the real content. A variable declared inside loop() is destroyed and recreated fifty times a second, so it can never remember anything. Say explicitly that you will come back to this in Part 3.

Ask the class: Ask: if I want the robot to remember that the gate is open, where must that variable live? (At class level, outside every method.)

Watch out: This is the root cause of a huge share of beginner bugs. Do not rush it.

---

11 if asks a question. else is the other door.

4 min.

Say: The symbol table is the useful half. Read each one aloud in plain English. The one-equals versus two-equals distinction deserves ten seconds of emphasis.

Ask the class: Ask: read this out in English — if (gamepad1.a && !gateOpen). (If A is pressed and the gate is not open.)

---

12 while repeats. for counts. Use both sparingly.

4 min.

Say: The teaching point is restraint, not syntax. Most beginners reach for a while loop to wait for things, and that is exactly the wrong instinct in FTC. Plant the seed here; Part 3 makes it a rule.

Ask the class: Ask: what would happen if I wrote 'while the arm is not up, do nothing'? (The whole robot stops responding.)

---

13 switch picks one option out of many.

New in v2. 5 min.

Say: Introduce switch as a language feature here, on a neutral example, so that Part 3 can use it for state machines without teaching two new things at once. The missing-break bug is worth demonstrating on the board.

Ask the class: Ask: what happens if level is 7? (default runs.) What if I delete the break after case 1? (It runs case 2 as well.)

Watch out: Keep the example non-robot and simple. The robot application comes in Part 3.

---

14 An enum is a short list of allowed names.

New in v2. 5 min.

Say: The selling point is that the compiler catches your typo at your desk instead of at a competition. Connect back to slide 7: this is the enum-inside-a-class exception you promised.

Ask the class: Ask: invent an enum for our shooter. What are the allowed names? (Accept anything sensible — it previews Part 3.)

Watch out: Students sometimes think an enum is a number. It is not; it is a name. Do not let the int comparison muddy that.

---

15 A method is a machine: things go in, a thing comes out.

5 min.

Say: Walk the five parts of the signature in order, pointing at each on the code. A juicer is a good physical analogy: fruit in, juice out, and you do not care what happens inside.

Ask the class: Ask: what goes in and what comes out of a method called isGateOpen()? (Nothing in; a boolean out.)

Watch out: void confuses people. Say it means 'this machine does something but hands nothing back'.

---

16 public and private: who is allowed in your room?

4 min.

Say: Three keywords only. In practice rookies need exactly two: private for everything inside a subsystem, public for the handful of methods others call. protected is here for recognition, not use.

Ask the class: Ask: our Shooter class has a flywheel motor and a fire() method. Which should be private and which public? (Motor private, fire() public.)

Watch out: This slide is the foundation for encapsulation in Part 2. Say so.

---

17 static means one copy for everybody.

4 min.

Say: Math.toDegrees is the cleanest example: everyone has used a static method without knowing it. Do not go deep. Recognition is enough at this stage.

Ask the class: Ask: have you ever made a Math object? (No — that is the point.)

Watch out: If a student asks why static exists at all, 'so you do not have to make an object just to do arithmetic' is a good enough answer today.

---

18 final means: write it once, then it is frozen.

3 min.

Say: Frame final as documentation that the compiler enforces. The 11pm line usually gets a laugh and it is also true.

Ask the class: Ask: name one number in our robot that should never change during a match.

---

19 An @ is a sticky note for the compiler.

4 min.

Say: The forgotten @TeleOp is the single most common first-week support call. Tell them the symptom in advance: 'my program is not in the list'. Then they will self-diagnose instead of asking.

Ask the class: Ask: your OpMode is not showing up on the Driver Station. What is the first thing you check? (@TeleOp or @Autonomous.)

Watch out: @Override will make more sense after Part 2. Mention that it is coming back.

---

20 Sixteen words you now own.

Recap. 3 min. Good natural break point.

Say: Same list as slide 6, different question. Go through it and ask the room to supply the right-hand column from memory before you reveal it.

Ask the class: Ask: cover the right column. Can you explain each word in one sentence?

Watch out: This is the end of session one if you are splitting the deck across meetings.

Part 2 — Object oriented concepts

21 Section opener — Object oriented concepts

Section opener. 2 min.

Say: The subtitle is the definition — read it out. Say that this part uses cars and animals, not robots, on purpose: the ideas are easier to see in things they already understand, and the robot comes back at the end.

Watch out: Do not let anyone leave this section thinking OO is a Java thing. It is a way of thinking that happens to suit Java.

---

22 Ten words you are about to meet.

Preview. 2 min.

Say: Read the list. Do not explain. Tell them these ten words come up in every software job interview for the rest of their lives, which is true and tends to focus minds.

Watch out: Two minutes.

---

23 Everything around you is already an object.

The foundation of the section. 6 min. Take your time.

Say: Everything in this part is built on 'knows things, can do things, keeps its own state'. Say that sentence three times during the section. The two columns are deliberately the same shape so students see the pattern rather than the examples.

Ask the class: Ask: pick any object in this room and tell me one thing it knows and one thing it can do. Go round four or five students. A door, a phone, a water bottle all work.

Watch out: If students only get one idea from Part 2, it should be this slide. Do not skip it to save time; skip something later instead.

---

24 A class is the blueprint. An object is the actual thing.

5 min.

Say: The blueprint-versus-house line is the one students remember. Stress that the three vehicles share a design but not their details — schoolBus being yellow does not make myCar yellow.

Ask the class: Ask: if I change the colour of myCar, does schoolBus change? (No.) If I add a horn to the Vehicle class, do all three get one? (Yes.)

Watch out: Recipe-and-cake works too, but a vehicle carries better into the inheritance slide later, so stay with vehicles.

---

25 Abstraction: you drive a car without opening the bonnet.

5 min.

Say: The washing machine row is the strongest: a hugely complex machine reduced to one dial. Push the point that abstraction is a design decision — somebody chose what to put on the dashboard.

Ask the class: Ask: how many buttons does a microwave really need? Then: if our Shooter class had twenty public methods, would that be a good dashboard?

Watch out: Students confuse abstraction with encapsulation. Abstraction is about *what you choose to show*; encapsulation is about *keeping the rest safe*. Next slide.

---

26 Encapsulation: the bonnet is bolted shut for a reason.

5 min.

Say: Make the distinction from the previous slide explicit: abstraction chose the dashboard, encapsulation is what stops you bypassing it. The 'maker could never change anything' point is the real engineering reason and worth saying slowly.

Ask the class: Ask: if our Intake class made its motor public and five other files grabbed it directly, what happens when we swap that motor? (You have to fix five files.)

Watch out: Tie it back to slide 16 by name. Students should feel private and public being reused, not newly introduced.

---

27 Inheritance: a child class gets its parent's stuff for free.

6 min.

Say: Run the vehicle tree, then run the animal tree verbally as a second example — Animal knows how to eat and breathe, Mammal adds warm blood, Dog adds barking. Two examples make the pattern visible rather than the example memorable.

Ask the class: Ask: build me a tree for something else. Shapes, or instruments, or phones. Let them argue about where things sit; the arguing is the learning.

Watch out: Someone will ask about multiple parents. Java does not allow a class to extend two classes. Say that and move on — interfaces are out of scope today.

---

28 Say the sentence out loud. Whichever sounds right, wins.

5 min.

Say: This is the practical test students will actually use. Make them say the sentences aloud — the wrong one sounds obviously silly, which is the whole point of the trick.

Ask the class: Ask, rapid fire: a Shooter and a Motor? (has-a.) A Dog and an Animal? (is-a.) A House and a Door? (has-a.) A Square and a Shape? (is-a.)

Watch out: The 'start with has-a' advice is worth repeating. Over-eager inheritance is the commonest way a beginner's design goes wrong.

---

29 Now the same five ideas, on real robot code.

The payoff slide. 6 min.

Say: Walk the right-hand table one row at a time and point at where it appears in the diagram. Students should recognise every word — nothing new is introduced here. If a row draws a blank, that is your signal to revisit that concept.

Ask the class: Ask: which of the five ideas is the red text showing? (Encapsulation.) Which is the green text? (Abstraction — the dashboard.)

Watch out: This is the slide that justifies the whole section. Do not rush it even if you are running late.

---

30 Ten words you now own.

Recap. 3 min.

Say: Cover the right column and work through it. Then ask for a one-sentence definition of object oriented programming in their own words. Anything close to 'building software out of parts that know things and do things' is a pass.

Ask the class: Ask: explain encapsulation to someone who has never programmed. If they reach for the bonnet, it has landed.

Watch out: Natural break point if you are splitting across meetings.

Part 3 — Thinking first

31 Section opener — Thinking first

Section opener. 1 min.

Say: Tell them this is the part experienced FTC programmers say they wish they had learned first. It is also the part with the least typing.

---

32 Ten words you are about to meet.

Preview. 2 min.

Say: Read and move on.

---

33 Draw it before you type it. Paper is cheaper than debugging.

4 min. Make this interactive.

Say: Draw the morning routine on the whiteboard with the class calling out the next shape. Four shapes is genuinely all there is, so they can be fluent in five minutes.

Ask the class: Ask: what shape is 'is it raining?' (Diamond.) What shape is 'put on shoes'? (Rectangle.)

Watch out: Do not introduce any other flowchart symbols. Four is enough and more is clutter.

---

34 Same four shapes. Real job: score one game piece.

4 min.

Say: The 'no' loop is the teaching point, not the flowchart. Trace it with a finger: piece not held, go back, check again next loop. Connect it to slide 3 — each trip round that loop is one pass of the robot's fifty-per-second cycle.

Ask the class: Ask: what is the robot doing while it waits for the piece? (Still driving, still reading the gamepad, still sending telemetry.)

---

35 An event is just: something happened, and we care.

New in v2. 5 min.

Say: The key idea is that the word 'event' gets used constantly but rarely defined. Define it plainly, then deliver the FTC-specific twist: there are no interrupts here, your loop must poll. This is why loop() exists at all.

Ask the class: Ask: name three events that happen in a match. (Driver presses a button, a sensor sees a piece, the 30-second auto ends.) Then: who tells our program about them? (Nobody — it has to look.)

Watch out: Students coming from app-building or Scratch expect 'when button clicked' blocks. Explicitly say FTC does not work that way.

---

36 Holding a button down is not one event. It is fifty.

New in v2. 5 min. This slide earns its place.

Say: Act it out: hold your hand up and say 'true, true, true, true' fast. That is what the loop sees. Then say 'the only interesting moment was the first one'. The phrase to leave them with is 'react to the change, not the state'.

Ask the class: Ask: our gate toggles on button A. I hold A for one second. How many times does it toggle if we get this wrong? (About fifty.)

Watch out: This sets up the very next slide perfectly — the toggle example is the fix. Make the handover explicit: 'so how do we remember? That is state.'

---

37 State is remembering what you already did.

6 min. Trace it by hand.

Say: Walk four loops on the whiteboard as a table: gamepad1.a, aAlreadyPressed, motorOn. Loop 1 not pressed, loop 2 pressed, loop 3 still pressed, loop 4 released. Students see the toggle fire exactly once. This is the payoff for slides 10 and 36 together.

Ask the class: Ask: on loop 3 the button is still held. Why does nothing happen? (Because aAlreadyPressed is now true, so the change is over.)

Watch out: Doing the trace by hand is worth more than any explanation. Budget the time.

---

38 A traffic light is a state machine. So is your shooter.

5 min.

Say: Everyone has watched a traffic light, so use it hard. Emphasise 'exactly one state' — a light is never green and red together, and neither is a subsystem. Note the light does not block: it checks the clock each cycle, exactly like loop().

Ask the class: Ask: could we add a flashing-amber state? What would the arrows in and out be? (Good test of whether they have got transitions and triggers.)

---

39 enum and switch, doing a real job together.

6 min.

Say: Nothing here is new — enum came in slide 14, switch in slide 13, state machines in slide 38. Say that out loud; it is reassuring. The enum-is-circles, switch-is-arrows mapping is the line to repeat.

Ask the class: Ask: point at the line that takes an arrow. (state = State.WAIT_FOR_SENSOR_RELEASE.) Point at the line that does the work. (setServoPosition.)

Watch out: Two-things-per-case is the rule that keeps their code clean all season. Write it on the board.

---

40 Our shooter, drawn the same way.

5 min.

Say: Same circles as the traffic light on purpose — say so, so the continuity registers. Have a student read the four states aloud as a sentence. Then point out that this drawing is the design document; the Java is transcription.

Ask the class: Ask: what is the trigger on the arrow from SPINNING UP to READY? (Flywheels reach target speed.) What would happen if we had no such check? (We would fire before the wheels were fast enough and the shot would fall short.)

Watch out: If a student asks what happens when the shot fails midway, that is a great question — the honest answer is 'we would add another state and another arrow'.

---

41 Never make the loop wait.

5 min. The rule they must leave with.

Say: State it as a rule, not a suggestion. Everything in Parts 0, 3 and 4 points here. Say the consequence in match terms: a frozen loop means a driver pushing sticks and nothing happening while the opponent scores.

Ask the class: Ask: our gate takes two seconds to open. How do we wait for it without freezing? (A state that checks a timer each loop.)

Watch out: If you only have time for one slide from this section at a revision session, it is this one.

---

42 Ten words you now own.

Recap. 3 min.

Say: Cover and quiz. Natural break point.

Part 4 — The FTC SDK

43 Section opener — The FTC SDK

Section opener. 1 min.

Say: Mostly vocabulary plus one big rule about names. Tell them this part is the one they will reopen most often during build season.

---

44 Eleven words you are about to meet.

Preview. 2 min.

Say: Read and move on.

---

45 The SDK is your translator.

4 min.

Say: Call back to abstraction explicitly — the SDK is the biggest dashboard-over-machinery example they will meet. The import lines finally make sense here, so point that out.

Ask the class: Ask: which layer do we write? (The top one only.) Which layer would we have to write if the SDK did not exist? (All of them.)

---

46 An OpMode is one program. init and loop are its heartbeat.

5 min.

Say: Map the diagram onto the physical buttons on the Driver Station — INIT, PLAY, STOP. If you have a Driver Station in the room, hold it up and press them. The loop box is the same loop from slide 3; say so.

Ask the class: Ask: where would you put the code that closes the gate before the match starts? (init().) Where does the driving code go? (loop().)

Watch out: Only two methods are required. Do not overload them with the optional three — recognition is enough.

---

47 init() puts everything back to normal.

3 min.

Say: A short slide about a habit rather than a feature. The competition-pressure framing is what makes it stick — describe the scene of a match ending badly because a counter was left over from the last run.

Ask the class: Ask: name two things our robot should reset in init().

---

48 hardwareMap is the robot's contact list.

5 min. Demonstrate if you possibly can.

Say: If a Control Hub and Driver Station are available, make a config entry live, then deliberately mistype the name in code and show the crash. Two minutes of that is worth twenty minutes of talking.

Ask the class: Ask: my config says left_front and my code says leftFront. Will it work? (No — and the error message will say so.)

Watch out: Teach them to read the exception text. Rookies see red and panic; the message usually names the exact device.

---

49 Motors take power. Servos take position.

4 min.

Say: Power versus position is the distinction that matters. A motor spins while you tell it to; a servo goes somewhere and stays. The encoder note connects straight back to the shooter state machine on slide 40.

Ask the class: Ask: the gate uses a servo. Does setPosition(0.3) mean 'open slowly' or 'go to this spot'? (Go to this spot.)

---

50 Buttons are true or false. Sticks are decimals.

4 min.

Say: Hand a gamepad round if you have one. The negative-y quirk is the thing to say twice — it will otherwise cost somebody an evening. Link the last line back to polling on slide 35 so the concepts join up.

Ask the class: Ask: I push the left stick all the way forward. What number do I get? (-1.0.) Why the minus? (Historical quirk — just remember it.)

Watch out: If a student asks 'why is up negative', the honest answer is that it is a convention inherited from screen coordinates. Do not invent a reason.

---

51 Telemetry is how the robot talks back to you.

4 min.

Say: 'Print your state' is the single most useful debugging habit in FTC. Make it a team rule here. Point out that slide 40's four state names are exactly what they should expect to see cycling on the screen.

Ask the class: Ask: the shooter never fires. Telemetry says State: SPINNING_UP and it never changes. What is wrong? (The flywheels are not reaching the target speed, so that transition never fires.)

Watch out: That question is the whole point of the slide. Give them time to work it out.

---

52 A dashboard lets you watch numbers and turn knobs, live.

3 min.

Say: Be straightforward that the team has not decided. Modelling 'we will test and then choose' is a better lesson than pretending to have an answer. Update this slide once the decision is made.

Ask the class: Ask: why is changing a number without rebuilding worth so much? (A rebuild and redeploy is minutes; tuning needs dozens of tries.)

---

53 Eleven words you now own.

Recap. 3 min.

Say: Cover and quiz.

Part 5 — Closing the loop

54 Section opener — Closing the loop

1 min.

Say: Signpost that the promise from slide 4 is about to be paid off.

---

55 Remember slide 4? Read it now.

The payoff. 6 min. Do not rush.

Say: Do not read the table yourself. Point at each line of code and ask the room to explain it. They can. Reveal the table row only after somebody has said it. The feeling of 'I could not read this an hour ago' is the whole purpose of the deck.

Ask the class: Ask: line by line, what does this do? Take volunteers for each.

Watch out: If the room goes quiet on a line, that tells you which part to revisit next session. Note it down.

---

56 Cheat sheet — one sentence from each part.

2 min.

Say: Literally tell them to photograph it. Five sentences is a realistic amount to carry away from one session.

---

57 Three jobs this week. All of them small.

3 min.

Say: Insist on typing rather than copy-pasting; muscle memory for syntax errors is part of the learning. Task 3 needs no computer, so nobody is blocked by laptop access. The green box is a team norm worth enforcing from now on.

Ask the class: Ask: what are the three things you bring when you ask for help?

Watch out: Set a specific deadline and a specific person to show it to, or it will not happen.

---

58 Everything cited in this deck.

2 min.

Say: Point them at the free book specifically; it is the single best follow-up and costs nothing. The date-check rule matters: FTC changes every year, and libraries get rewritten.

Watch out: Hand out or share the PDF at this point, not before — otherwise they read ahead instead of listening.